



Documentación del SaaS

10742A-P26 SISTEMAS DISTRIBUIDOS

José Luis Álvarez Mánica

José Espinoza Ordas

Fernando Agustin Hernández Rivas

20 mayo del 2026

Primavera 2026

Tabla de contenido

Nombre del Startup	1
Integrantes y Roles propuestos	1
Nombre del SaaS	1
Problema que resuelve	1
Usuarios Principales	1
Descripción general de la solución	2
Flujo principal.....	2
Requerimientos funcionales iniciales	5
Requerimientos no funcionales	7
Módulos principales del producto	8
Servicios Backend.....	9
Entidades principales.....	10
Eventos del sistema	12
Uso de Redis	13
Uso de RabbitMQ.....	13
Wireframes.....	14
Arquitectura	19
Alcance del MVP	21
Riesgos técnicos.....	21
Historias de usuario	22
Backlog.....	23
Endpoints.....	24
Pruebas.....	25
Fallas simuladas.....	25
Repositorio oficial.....	26
Página web en Vercel.....	26

Nombre del Startup

Overview

Integrantes y Roles propuestos

Jose Luis Alvarez Manica -- Backend Developer / QA Engineer / DevOps

Jose Espinoza Ordas -- Arquitecto de software / Data

Fernando Agustin Hernandez Rivas -- FrontEnd Developer

Nombre del SaaS

loteur (iot + moniteur)

Problema que resuelve

Adentrarse en el mundo del IoT como usuario independiente o entusiasta implica una curva de aprendizaje considerable. Quienes desean construir sus propios dispositivos en lugar de usar soluciones comerciales cerradas se enfrentan rápidamente a desafíos que van más allá del hardware: ¿cómo registro mis dispositivos? ¿cómo sé si están activos? ¿dónde van los datos que envían?

Resolver cada una de estas preguntas por separado requiere conocimientos de redes, bases de datos y protocolos de comunicación, lo que desalienta a muchos antes de llegar a lo que realmente les interesa: construir y experimentar. Esta plataforma busca eliminar esa barrera ofreciendo una base lista para usar, para que el foco del usuario esté en crear, no en resolver infraestructura.

En corto: brindar una Plataforma sencilla para facilitar a creadores de dispositivos IoT el registro, manejo y analisis de los datos recopilados.

Usuarios Principales

Administrador: Usuario con acceso completo a la plataforma, responsable de su configuración y supervisión general. Puede gestionar las cuentas de los usuarios registrados, monitorear el estado global del sistema y atender situaciones que escapen al control del usuario regular.

Usuario: Entusiasta o desarrollador independiente que utiliza la plataforma para registrar y administrar sus propios dispositivos IoT. Puede dar de alta nuevos dispositivos, consultar los datos que estos envían, revisar su estado en tiempo real y configurar notificaciones según sus necesidades.

Descripción general de la solución

loteur es una plataforma SaaS de microservicios diseñada para que entusiastas y desarrolladores independientes puedan registrar, monitorear y analizar sus dispositivos IoT sin necesidad de conocimientos en infraestructura. El sistema recibe datos de los dispositivos vía HTTP en formato JSON, valida su identidad contra Redis para minimizar latencia, y desacopla la recepción del procesamiento mediante colas en RabbitMQ, persistiendo los registros en MongoDB por su esquema flexible. La arquitectura distribuye responsabilidades entre servicios especializados: autenticación con JWT, gestión de dispositivos en PostgreSQL, telemetría agrupada por intervalos y notificaciones por correo ante eventos relevantes como desconexiones o umbrales superados.

El flujo principal abarca desde el registro del usuario y el alta de sus dispositivos (identificados por dirección MAC y UUID), hasta la consulta en tiempo real de su estado operativo y la generación de reportes de telemetría. Un API Gateway centraliza el acceso externo, mientras que Redis actúa como caché de validaciones rápidas y lista negra de tokens revocados, y RabbitMQ coordina la comunicación asíncrona entre los microservicios internos.

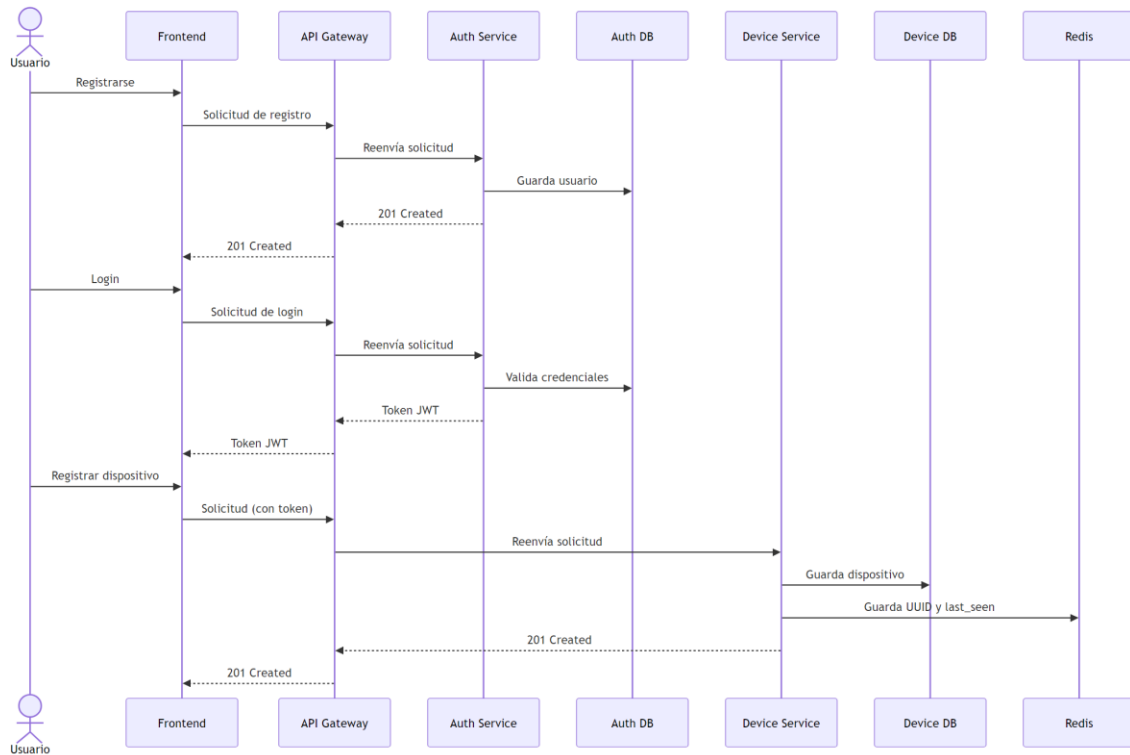
Flujo principal

Flujo 1: Registro de usuario, login y alta de dispositivo

El usuario accede al frontend e introduce sus datos para crear una cuenta. El frontend envía la solicitud al API Gateway, que la redirige al Auth Service. Este valida los datos, persiste el nuevo usuario en su base de datos y responde con un 201 Created que se propaga de vuelta al frontend.

Para iniciar sesión, el usuario envía sus credenciales. El Auth Service las valida contra su base de datos y, si son correctas, genera y retorna un token JWT. Este token deberá incluirse en todas las peticiones posteriores para autenticar al usuario.

Una vez autenticado, el usuario puede registrar un dispositivo proporcionando su dirección MAC y un nombre. El Device Service persiste el dispositivo en su base de datos relacional (PostgreSQL) y almacena en Redis únicamente la información mínima necesaria para validaciones rápidas: el UUID del dispositivo y su último tiempo visto (last_seen).



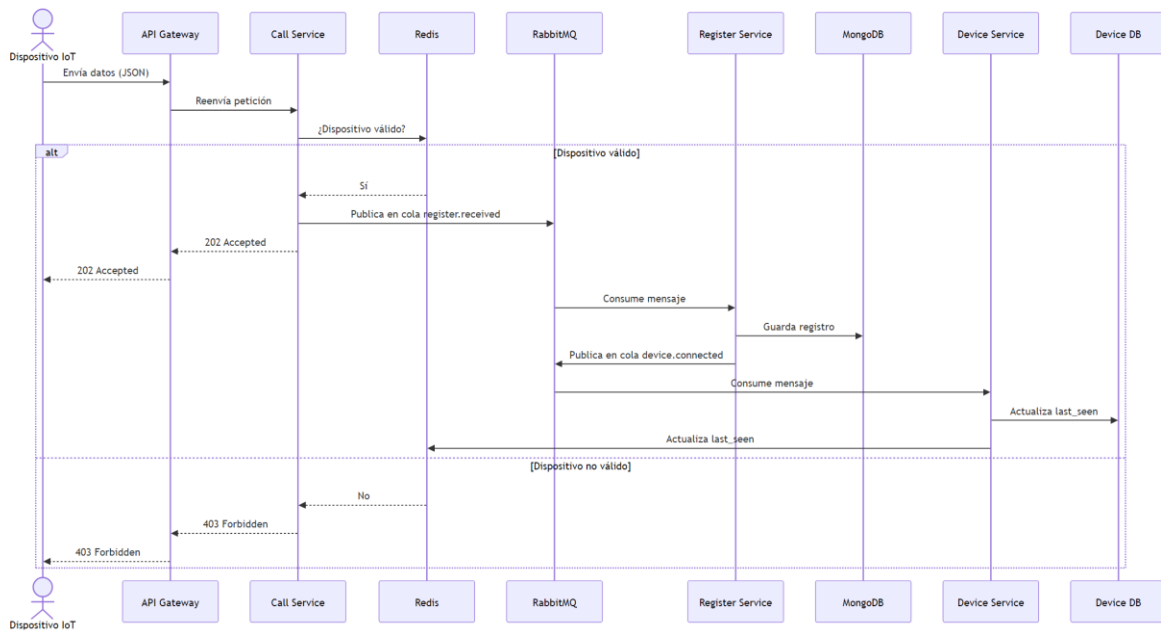
Flujo 2: Envío de datos desde un dispositivo IoT

Un dispositivo envía su payload en formato JSON al API Gateway mediante HTTP. El Gateway redirige la petición al Call Service, que consulta Redis para verificar si el UUID del dispositivo existe y está activo. Esta consulta es intencional mente ligera para no introducir latencia en el camino crítico.

Si el dispositivo es válido, el Call Service publica el mensaje en la cola `register.received` de RabbitMQ y responde con `202 Accepted`, desacoplando así la recepción del procesamiento. El Register Service consume el mensaje de la cola y persiste el registro en MongoDB, cuyo esquema flexible permite admitir distintos formatos de payload según el dispositivo.

Una vez guardado el registro, el Register Service publica un evento en la cola `device.connected` con el UUID y el timestamp del mensaje. El Device Service consume este evento y actualiza el `last_seen` tanto en su base de datos relacional como en Redis, manteniendo ambas fuentes sincronizadas.

Si el dispositivo no existe en Redis o está marcado como inactivo, el Call Service rechaza la petición con un `403 Forbidden` sin llegar a encolar ningún mensaje.

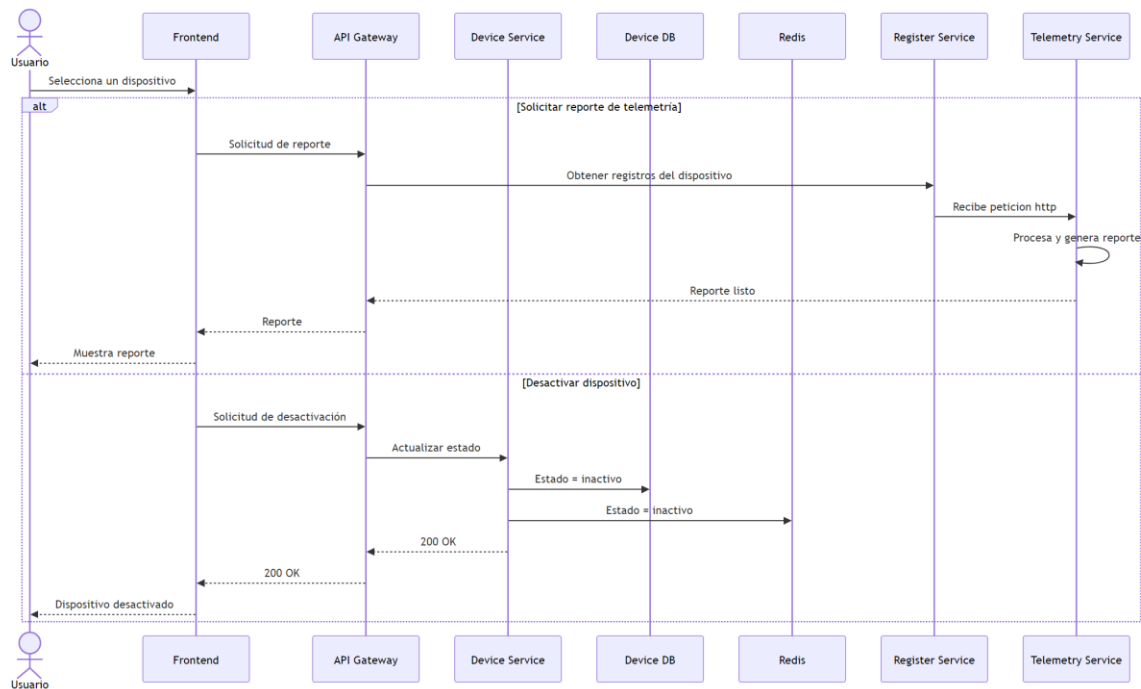


Flujo 3: Reporte de telemetría y desactivación de dispositivo

El usuario accede al frontend, selecciona uno de sus dispositivos y puede realizar dos acciones sobre él.

Si solicita un reporte, el API Gateway recupera los registros almacenados en el Register Service y realiza una petición http al servicio de telemetría. El Telemetry Service recibe los contenidos, procesa y analiza los datos, genera el reporte en su propia instancia de MongoDB y lo retorna al frontend para que el usuario pueda consultarlo.

Si el usuario marca el dispositivo como inactivo, el Device Service actualiza su estado tanto en la base de datos relacional como en Redis. A partir de ese momento, cualquier petición HTTP proveniente de ese dispositivo será rechazada por el Call Service al consultar Redis, sin necesidad de llegar a procesar el payload.



Requerimientos funcionales iniciales

RF-01 Registro y autenticación de usuarios

El sistema debe permitir crear una cuenta con credenciales básicas y autenticar sesiones mediante tokens JWT.

Enfoque abordado: Autenticación

RF-02 Gestión de roles de usuario

El sistema debe diferenciar entre rol Administrador (acceso total, gestión de cuentas) y Usuario regular (gestión de sus propios dispositivos).

Enfoque abordado: Autenticación

RF-03 Registro de dispositivos IoT

El usuario puede dar de alta un dispositivo proporcionando su dirección MAC y un nombre. El sistema genera un UUID y lo persiste en PostgreSQL.

Enfoque abordado: Dispositivos

RF-04 Métodos de conexión

El sistema puede obtener mediciones de dispositivos solamente mediante HTTP.

RF-05 Recepción de datos desde dispositivos IoT

El sistema debe aceptar payloads JSON, desde dispositivos previamente registrados, validar su identidad y encolar el mensaje en RabbitMQ con respuesta 202 Accepted.

Enfoque abordado: Datos

RF-06 Almacenamiento flexible de registros

Los datos recibidos de los dispositivos deben persistirse en MongoDB con los campos que el usuario decida, el registro se guarda como documento dentro de la carpeta del dispositivo.

Enfoque abordado: Datos

RF-07 Monitoreo de estado en tiempo real

El usuario puede consultar el estado operativo de cada dispositivo (activo/inactivo) y su último tiempo visto (last_seen), mantenido en Redis.

Enfoque abordado: Dispositivos

RF-08 Detección automática de desconexión

Si un dispositivo supera 2–3 plazos de tiempo, según el dispositivo, sin enviar datos el sistema debe marcarlo como desconectado (evento device.disconnected) y notificar al usuario.

Enfoque abordado: Dispositivos

RF-09 Generación de reportes de telemetría

El usuario puede solicitar un reporte de un dispositivo. El Telemetry Service procesa los registros almacenados y genera análisis agrupados por intervalos de tiempo en su propia instancia MongoDB.

Enfoque abordado: Telemetría

RF-10 Notificaciones por correo electrónico

El sistema debe enviar alertas por email ante eventos relevantes (dispositivo desconectado, umbral superado). Cada intento queda registrado con su resultado (éxito/fallo) y un mecanismo de reintento vía RabbitMQ.

Enfoque abordado: Notificaciones

RF-11 Registro de eventos y errores del sistema

El sistema debe mantener un historial de eventos relevantes clasificados por nivel de severidad (evento system.error). El administrador puede consultar este log desde la plataforma.

Enfoque abordado: Administración

RF-12 Gestión de cuentas por el administrador

El administrador puede consultar, suspender o eliminar cuentas de usuarios registrados y monitorear el estado global del sistema desde un panel dedicado.

Enfoque abordado: Administración

Requerimientos no funcionales

RNF-01 Baja latencia en el camino crítico de recepción

La validación de un dispositivo entrante (consulta a Redis + encolado en RabbitMQ) debe completarse con latencia mínima para no bloquear el flujo de datos. El diseño intencional usa Redis precisamente para evitar roundtrips a la base de datos relacional.

Enfoque abordado: Rendimiento

RNF-02 Seguridad y control de acceso

Todas las peticiones al API Gateway deben validarse mediante token JWT. Los dispositivos sin UUID registrado o marcados como inactivos reciben 403 Forbidden.

Enfoque abordado: Seguridad

RNF-03 Consistencia de datos entre distintos motores de bases

El sistema debe garantizar la sincronización entre PostgreSQL y MongoDB (p.ej. last_seen). Ante fallos parciales, debe detectar y resolver desincronizaciones para evitar inconsistencias en el estado reportado de dispositivos.

Enfoque abordado: Operaciones

RNF-04 Escalabilidad horizontal de los microservicios

La arquitectura de microservicios debe permitir escalar horizontalmente el Call Service (principal cuello de botella) y los consumidores de RabbitMQ de forma independiente ante picos de concurrencia.

Enfoque abordado: Rendimiento

RNF-05 Usabilidad para creadores sin experiencia en infraestructura

La interfaz y los flujos de registro, alta de dispositivos y consulta de datos deben ser lo suficientemente simples para que un entusiasta de IoT pueda operar la plataforma sin conocimientos de redes, bases de datos ni protocolos.

Enfoque abordado: UX

Módulos principales del producto

Autenticación: Este módulo se encarga del registro de usuarios, el inicio de sesión y la gestión de sesiones activas. El Auth Service valida las credenciales del usuario contra su propia base de datos y, si son correctas, emite un token JWT que deberá incluirse en todas las peticiones posteriores. Adicionalmente, mantiene una instancia de Redis dedicada donde publica los tokens que deben invalidarse por cierre de sesión o renovación, funcionando como lista negra consultada por el API Gateway en cada solicitud entrante.

Dispositivos: Este módulo gestiona el ciclo de vida de los dispositivos IoT dentro de la plataforma. El Device Service permite dar de alta un dispositivo a partir de su dirección MAC y un nombre asignado por el usuario, generando un UUID que lo identifica de forma única. El estado operativo de cada dispositivo (activo, inactivo o eliminado) se persiste en PostgreSQL y se replica en Redis junto con su último tiempo de actividad (last_seen), de modo que el resto del sistema pueda realizar validaciones rápidas sin necesidad de consultar la base de datos relacional.

Datos: Este módulo cubre la recepción, validación y almacenamiento de los datos enviados por los dispositivos. El Call Service actúa como punto de entrada interno: consulta Redis para verificar si el dispositivo es válido y está activo, y de ser así publica el mensaje en la cola register.received de RabbitMQ respondiendo con un 202 Accepted, desacoplando la recepción del procesamiento. El Register Service consume esa cola y persiste el payload en MongoDB, cuyo esquema flexible permite admitir distintos formatos de datos según el dispositivo.

Telemetría: Este módulo se encarga de procesar y analizar los registros almacenados para generar reportes útiles al usuario. Cuando se solicita un reporte, el frontend realiza una llamada a través del API Gateway al Telemetry Service, que procesa las mediciones agrupándolas por intervalos de tiempo, almacena los resultados en su propia instancia de MongoDB y retorna el reporte al usuario.

Notificaciones: Este módulo es el responsable de mantener informado al usuario ante eventos relevantes del sistema. El Notification Service escucha eventos como device.disconnected o system.error y genera alertas por correo electrónico, registrando cada intento con su resultado (éxito o fallo). Para garantizar la entrega, utiliza RabbitMQ como mecanismo de reintento, de forma que los fallos en el envío no se pierdan y puedan procesarse nuevamente sin intervención manual.

Administración Este módulo centraliza las capacidades de supervisión y gestión del sistema, reservadas para el rol de administrador. Desde un panel dedicado, el administrador

puede consultar, suspender o eliminar cuentas de usuarios registrados y monitorear el estado global de la plataforma. Adicionalmente, el sistema mantiene un historial de eventos relevantes clasificados por nivel de severidad (info, warning y critical) que diversos servicios publican mediante el evento `system.error`, permitiendo al administrador consultar ese log directamente desde la plataforma.

Servicios Backend

- **API Gateway (API Intermediaria):** Actúa como punto de entrada único entre la interfaz web y el sistema interno, ocultando la arquitectura de los servicios al exterior. Se encarga de validar los tokens de acceso para permitir o denegar solicitudes a los distintos endpoints.
- **Auth Service (Servicio de autenticación):** Responsable del registro de usuarios y la gestión de sesiones mediante tokens. Mantiene una conexión con Redis para el manejo de tokens revocados o baneados y con una base de datos propia para el registro y manejo de usuarios.
- **Call Service (Servicio de llamadas):** API interna que recibe las solicitudes provenientes del API Gateway y las distribuye hacia los servicios correspondientes mediante colas de RabbitMQ con verificaciones rápidas de datos usando Redis. A su vez comprueba el estado de los dispositivos periódicamente en Redis y genera un evento para notificar al usuario por correo.
- **Device Service (Servicio de dispositivos):** Gestiona el registro y administración de los dispositivos, manteniendo un registro en Redis con los UUID de dispositivos autorizados. Utiliza PostgreSQL como base de datos dado que la estructura de los dispositivos es fija y predefinida.
- **Register Service (Servicio de registros):** Almacena los datos enviados por los dispositivos. Utiliza MongoDB como base de datos debido a la naturaleza variable de la información recibida.
- **Telemetry Service (Servicio de telemetría):** Procesa y analiza los datos almacenados para generar reportes. Cuenta con su propia instancia de MongoDB.
- **Notification Service (Servicio de notificaciones):** Envía notificaciones al usuario vía email ante eventos relevantes, como la caída de un dispositivo o la superación de un umbral en algún registro. Utiliza RabbitMQ como mecanismo de reintento en caso de fallos en el envío.

Entidades principales

Usuario: Tabla principal para la base de datos del servicio de autorización. Su propósito es almacenar la información mínima necesaria para identificar a cada usuario y determinar su rol dentro del sistema. Se propone el siguiente esquema:

User
PK id: UUID
+ name: VARCHAR(255)
+ email: VARCHAR(255)
+ password_hash: VARCHAR(60)
+ role: VARCHAR(20)
– user admin
+ created_at: TIMESTAMP

Dispositivo: Tabla principal para la base de datos del servicio de dispositivos. Su propósito es asociar cada dispositivo con el usuario correspondiente, registrando sus factores de identificación (dirección MAC y nombre asignado por el usuario), así como su estado operativo. A petición del cliente se agregaron los campos de grupo, icono y color para poder representar al dispositivo de manera visual en la plataforma y tener un control de que dispositivo pertenece a cual agrupación (se agrupación lógica, empresarial, geográfica, etc.).

Device
PK id: UUID
FK user_id: UUID
+ device_name: VARCHAR(255)
+ mac_address : VARCHAR(17)
+ report_interval: INTEGER (minutes)
+ status : VARCHAR(10)
– active inactive deleted
+ icon : VARCHAR(50)
+ color : VARCHAR(50)
+ group : VARCHAR(50)
+ last_seen: TIMESTAMP
+ created_at: TIMESTAMP

Registro: Esquema principal para la base de datos MongoDB del servicio de registros. Dado que el sistema debe admitir distintos formatos de datos, se le especificará al usuario que la

información proveniente de su dispositivo IoT debe enviarse en un JSON con un formato predefinido. Se propone el siguiente esquema:

Register
PK id: UUID FK device_id: UUID + created_at: TIMESTAMP + time_procesing: INTEGER (seconds) + values: JSON

Notificación del sistema: Tabla destinada al registro de los eventos relevantes detectados en el sistema. Su propósito es mantener un historial organizado de los logs con relevancia o impacto operativo, clasificados por nivel de severidad. Se propone el siguiente esquema:

System Notification
PK id: UUID + request_id: TEXT + service_name: TEXT + reason: TEXT + severity : VARCHAR(20) – info warning critical + message : TEXT + created_at: TIMESTAMP

Notificación de correo electrónico: Tabla destinada al registro de cada intento de notificación enviada al usuario a través de correo electrónico. Almacena la razón que originó el aviso, el nivel de severidad, el mensaje generado y el resultado del envío, indicando si este fue exitoso o fallido. Se propone el siguiente esquema:

Email Notification
PK id: UUID FK user_id: UUID FK device_id: UUID + email: VARCHAR(255) + reason: TEXT + severity : VARCHAR(20) – info warning critical + message : TEXT + status : VARCHAR(20) – sent unsent + created_at: TIMESTAMP

Telemetría: La tabla de telemetría (Telemetry) almacena los datos importantes generados por cada dispositivo a lo largo del tiempo, de manera periódica el servicio generará los reportes. Su propósito es generar la información necesaria para que un usuario pueda validar la utilidad de sus dispositivos IoT.

Telemetry	MetricData
PK id: UUID FK device_id: UUID + created_at: TIMESTAMP + metric_data: list[MetricData]	+ device_name: TEXT + value_list: list[str] + percentage_change: float + top_value: TEXT + most_frequent_value: TEXT

Eventos del sistema

- **device.register:** Call Service a Device Service, cuando un usuario registra un nuevo dispositivo.
- **device.connected:** Call service a Device Service, si se recibe un registro de un dispositivo se actualiza el tiempo del último mensaje recibido en Redis y en la base de datos correspondiente.
- **device.disconnected:** Call service a Notification Service, cuando un dispositivo deja de mandar información en un tiempo esperado (2 o 3 plazos de tiempo definidos), un trabajo periódico lo revisará en Redis y lo registrará como desconectado en Redis y en la base de datos correspondiente, luego procederá a activar este evento para notificar al usuario por correo electrónico.
- **system.error:** Diversos servicios a Notification Service, sirve para registrar los errores del sistema dentro de la base de datos.
- **register.received:** Call Service a Register Service, cuando se recibieron datos de un dispositivo se comprobará primero en Redis la pertenencia del dispositivo al sistema, si el dispositivo esta registrado se generara este evento para encolar su registro en la base de datos.

Uso de Redis

Redis se utilizará para los siguientes propósitos:

- **Gestión de tokens revocados:** Se contará con una instancia de Redis dedicada, accesible únicamente para el servicio de autorización y el API Gateway. En ella se publicarán los tokens de autorización que deban ser invalidados (ya sea por cierre de sesión o renovación) funcionando como una lista negra.
- **Caché interno:** Se dispondrá de una instancia de Redis de uso interno en la que el servicio de dispositivos registrará los dispositivos autorizados, identificados mediante un identificador único. Esta caché será consultada por el servicio de llamadas para determinar de forma eficiente si el registro de un dispositivo debe ser transmitido al servicio de registros o no.

Uso de RabbitMQ

RabbitMQ se utilizará como bróker de mensajes orientado a eventos, con las siguientes funcionalidades:

- **Enrutamiento entre servicios mediante tópicos:** El funcionamiento principal de RabbitMQ será gestionar mensajes a través de colas asociadas a llaves de enrutamiento específicas, facilitando la comunicación entre los distintos servicios. Por ejemplo, al recibir un registro y verificar el dispositivo, el servicio de llamadas publicaría en la cola correspondiente a la llave `device.register` para que el servicio de dispositivos lo consumiera. Las rutas propuestas y los servicios involucrados se describen en detalle en la sección de eventos.
- **Control de reintentos y gestión de colas:** Gracias al soporte de confirmaciones manuales (*ack/nack*), las colas de RabbitMQ pueden configurarse como un mecanismo de reintentos, permitiendo un control más robusto sobre el procesamiento y la entrega de mensajes. A su vez, en caso de caer un servicio RabbitMQ podrá guardar los eventos generados para ser consumidos cuando se recupere el servicio.

Wireframes

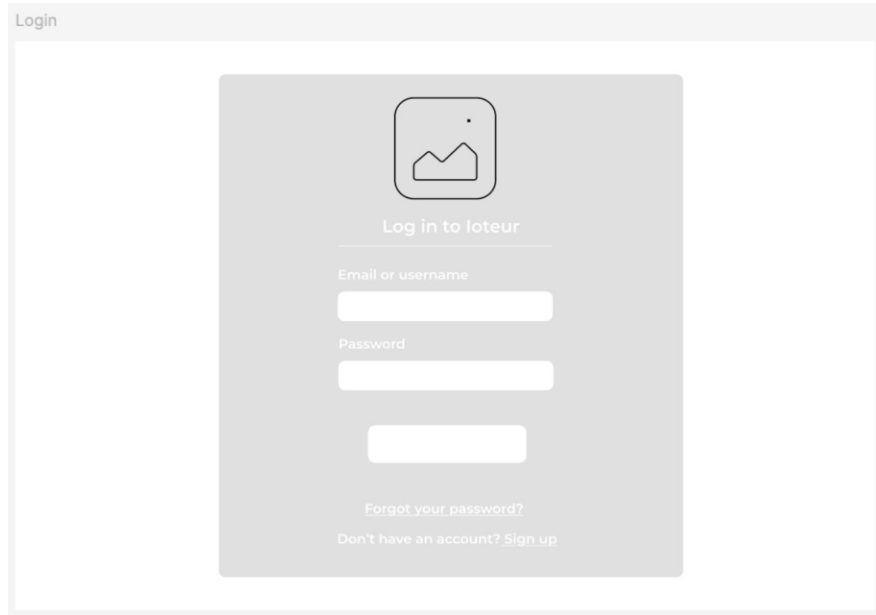


Figura 1. Inicio de sesión

Este wireframe representa la pantalla de autenticación del sistema **loteur**. Su objetivo es permitir que el usuario acceda de forma segura a la plataforma mediante sus credenciales.

Componentes principales

- **Logo de la aplicación**
 - Ubicado en la parte superior central.
 - Sirve para reforzar la identidad visual del sistema.
- **Formulario de inicio de sesión**
 - Campo para correo electrónico o nombre de usuario.
 - Campo para contraseña.
 - Botón principal para iniciar sesión.
- **Opciones adicionales**
 - Enlace de recuperación de contraseña (“Forgot your password?”).
 - Enlace para registro de nuevos usuarios (“Sign up”).

Funcionalidad esperada

- Validar credenciales del usuario.
- Redirigir al dashboard principal después del acceso exitoso.
- Permitir recuperación de acceso en caso de olvidar la contraseña.
- Facilitar el registro de nuevos usuarios.

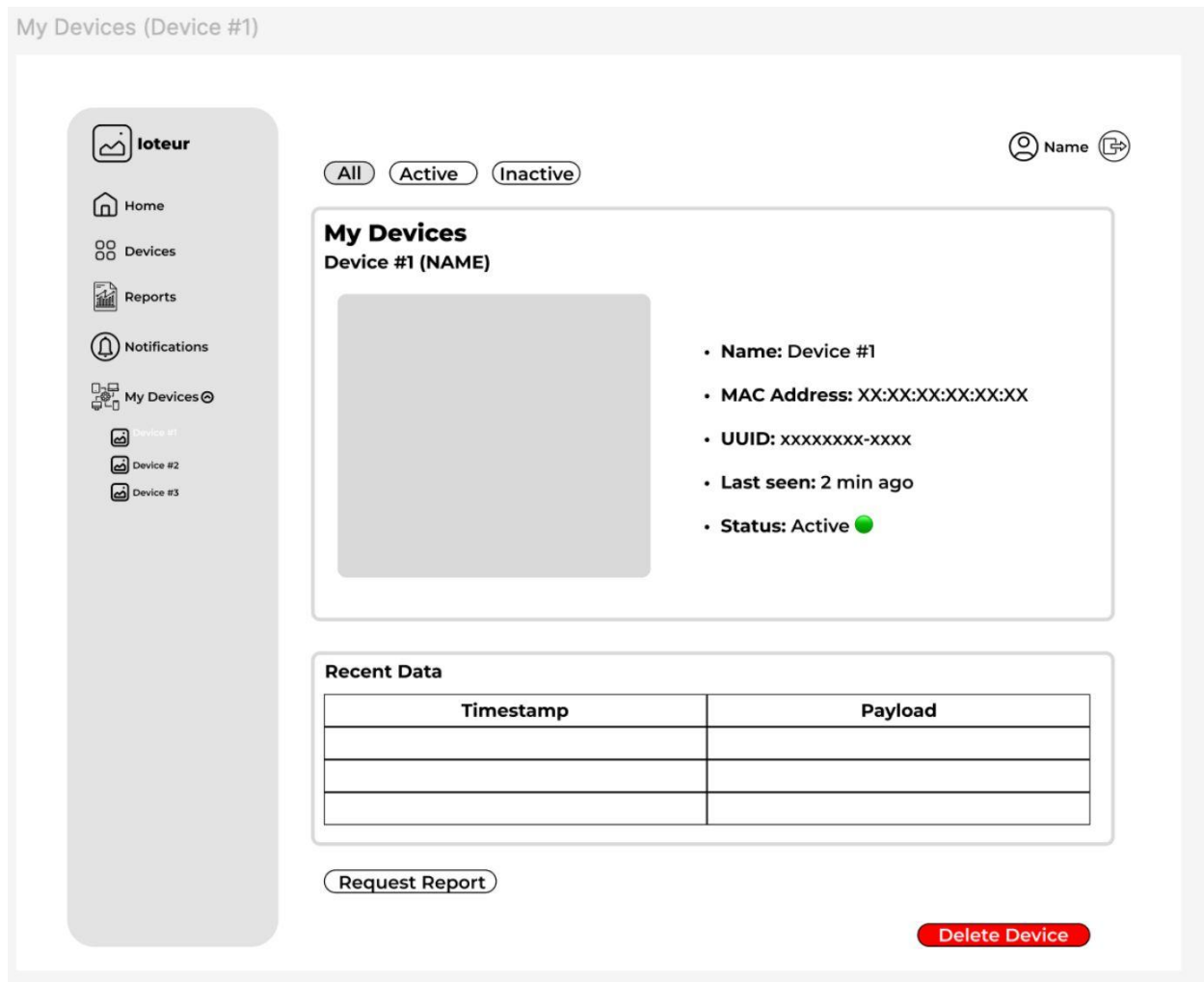


Figura 2. Vista detalle de un dispositivo seleccionado

Este wireframe muestra la pantalla de detalle de un dispositivo específico. Su función es proporcionar información técnica y operativa más completa del dispositivo seleccionado.

Componentes principales

Se muestran datos importantes como:

- Nombre del dispositivo.

- Dirección MAC.
- UUID.
- Última conexión.
- Estado actual (activo/inactivo).

Área visual del dispositivo

Espacio reservado para:

- Imagen del dispositivo.
- Vista previa.
- Gráfica futura.
- Indicadores visuales.

Tabla de “Recent Data”

Presenta los datos más recientes enviados por el dispositivo.

La tabla incluye:

- Timestamp (fecha y hora).
- Payload (datos enviados).

Botones de acción

“Request Report”

Permite generar o solicitar reportes relacionados con el dispositivo.

“Delete Device”

Permite eliminar el dispositivo del sistema.

Funcionalidad esperada

- Consultar telemetría reciente.
- Supervisar actividad del dispositivo.
- Generar reportes históricos.
- Eliminar dispositivos registrados.
- Verificar conectividad y estado en tiempo real.

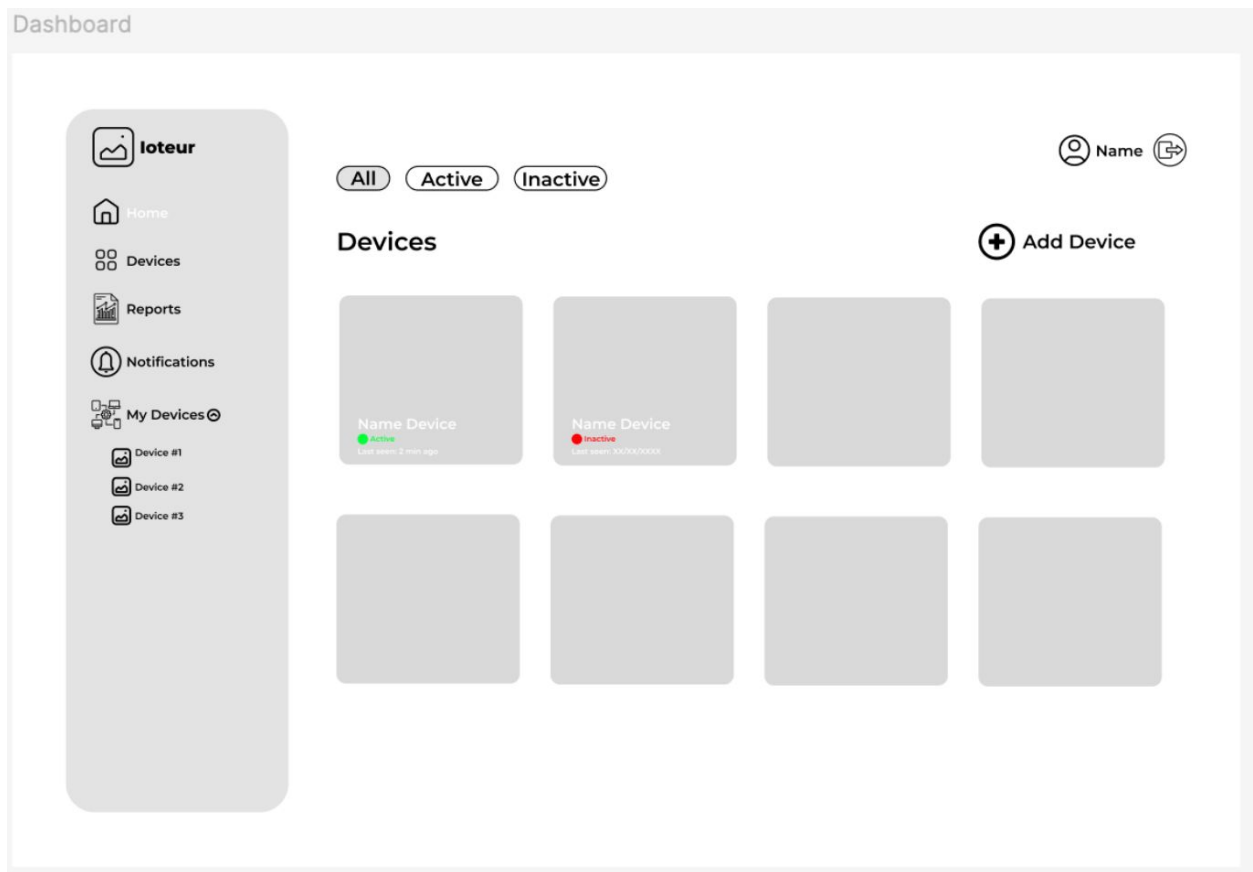


Figura 3. Panel de control de dispositivos

Este wireframe muestra la pantalla de detalle de un dispositivo específico. Su función es proporcionar información técnica y operativa más completa del dispositivo seleccionado.

Componentes principales

Información general del dispositivo

Se muestran datos importantes como:

- Nombre del dispositivo.
- Dirección MAC.
- UUID.
- Última conexión.
- Estado actual (activo/inactivo).

Área visual del dispositivo

Espacio reservado para:

- Imagen del dispositivo.
- Vista previa.
- Gráfica futura.
- Indicadores visuales.

Tabla de “Recent Data”

Presenta los datos más recientes enviados por el dispositivo.

La tabla incluye:

- Timestamp (fecha y hora).
- Payload (datos enviados).

Botones de acción

“Request Report”

Permite generar o solicitar reportes relacionados con el dispositivo.

“Delete Device”

Permite eliminar el dispositivo del sistema.

Funcionalidad esperada

- Consultar telemetría reciente.
- Supervisar actividad del dispositivo.
- Generar reportes históricos.
- Eliminar dispositivos registrados.
- Verificar conectividad y estado en tiempo real.

Arquitectura

El siguiente esquema presenta la arquitectura del MVP y la relación entre los componentes mínimos necesarios para que el sistema opere de extremo a extremo. El flujo inicia en el front-end, desde donde los usuarios interactúan con la plataforma. Las solicitudes viajan al API Gateway, encargado de centralizar el acceso, aplicar validaciones iniciales y redirigir cada petición al microservicio correspondiente.

La base funcional del sistema está compuesta por los microservicios internos. El primero es gestión de dispositivos, responsable del registro, configuración y estado de los equipos IoT. Después se encuentra registro de eventos, que recibe y conserva los eventos generados por los dispositivos. El microservicio de telemetría procesa esas mediciones y las almacena en una estructura de buckets dentro de MongoDB, agrupando datos por dispositivo y por intervalos de tiempo para optimizar almacenamiento y consultas históricas. Además de persistir la información, este servicio expone una API de consulta que permite recuperar lecturas recientes, series temporales y datos agregados para su visualización en la interfaz. Finalmente, el servicio de notificaciones atiende eventos relevantes y genera alertas cuando corresponde.

La comunicación interna entre estos componentes se apoya en RabbitMQ, lo que permite desacoplar el procesamiento y manejar tareas asíncronas sin bloquear el flujo principal. En cuanto al almacenamiento, cada microservicio accede únicamente a la base de datos asociada a su responsabilidad: PostgreSQL para dispositivos y usuarios, y MongoDB para eventos y telemetría. Como complemento, Redis funciona como capa de caché para acelerar consultas frecuentes relacionadas con usuarios y dispositivos.

El orden de los microservicios también define la secuencia recomendada de desarrollo y despliegue. La implementación debe realizarse de abajo hacia arriba y de izquierda a derecha, comenzando por la capa de persistencia y los microservicios que sostienen la lógica principal. Primero se construye la gestión de dispositivos, luego el registro de eventos, después la telemetría y finalmente las notificaciones. Una vez consolidado este núcleo, se integran los componentes de acceso externo (servicios de autenticación, API Gateway y front-end). Esta estrategia permite validar primero el comportamiento interno del sistema, reducir la complejidad inicial y habilitar un crecimiento incremental sobre una base estable.

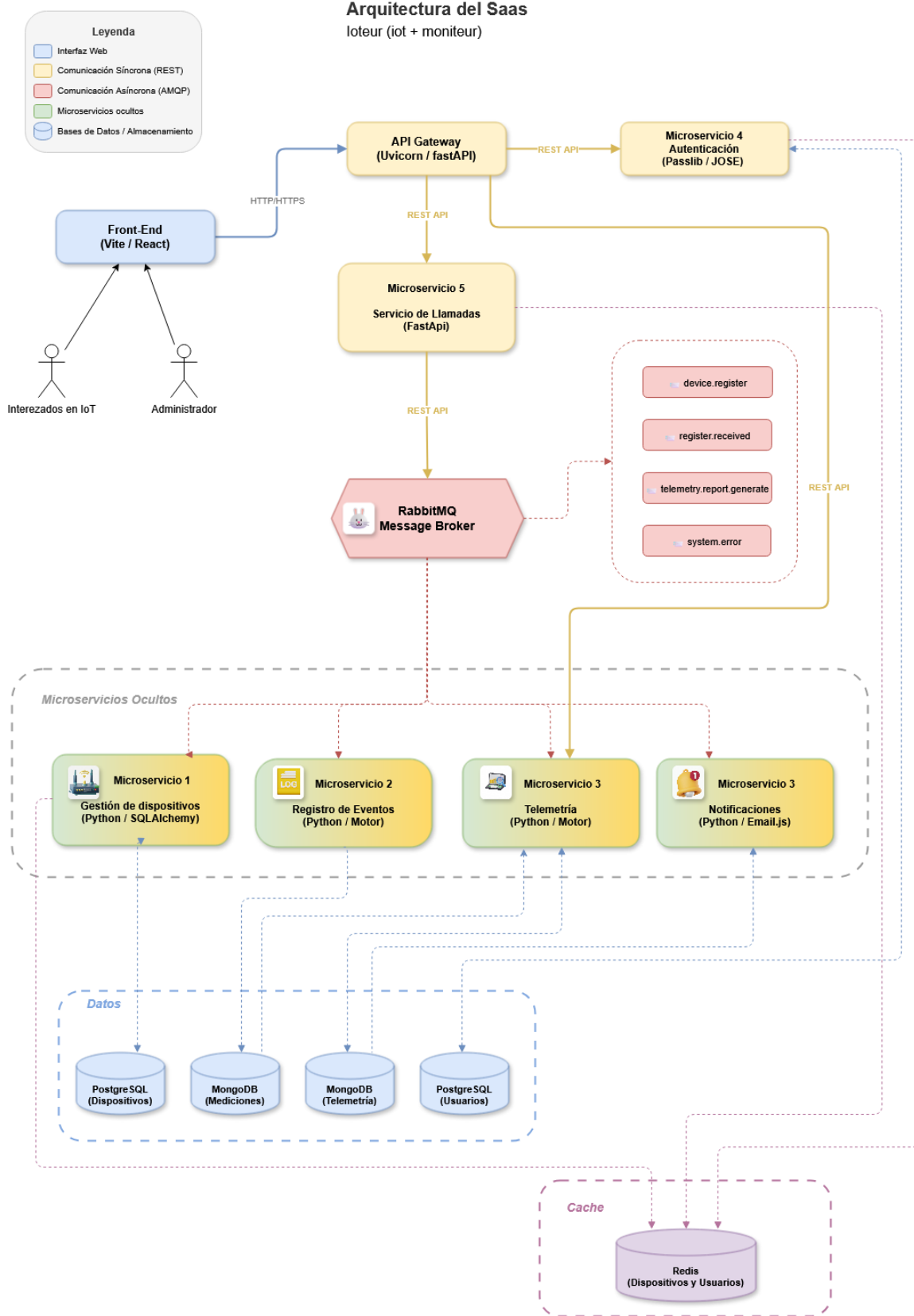


Figura 4. Arquitectura propuesta.

Alcance del MVP

El MVP consistirá en un prototipo funcional documentado y versionado en GitHub, diseñado para ejecutarse de forma local mediante Docker Compose. El sistema deberá poder levantarse en su totalidad siguiendo las instrucciones del README, incluyendo todos los servicios backend y el frontend.

En cuanto a funcionalidad, el prototipo cubrirá el flujo principal de extremo a extremo: registro e inicio de sesión de usuarios, alta y gestión de dispositivos IoT, recepción y almacenamiento de datos enviados por los dispositivos, y generación de reportes de telemetría. Adicionalmente, se contempla de forma tentativa realizar un despliegue en línea del sistema en Railway una vez consolidado el funcionamiento local.

Riesgos técnicos

1. **Consistencia entre Redis y PostgreSQL/MongoDB:** El sistema mantiene datos sincronizados entre Redis y las bases de datos persistentes (por ejemplo, el `last_seen` de un dispositivo). Ante una falla parcial (como que el evento `device.connected` se consuma, pero la escritura en PostgreSQL falle) ambas fuentes pueden quedar desincronizadas. Esto podría generar inconsistencias en el estado reportado de un dispositivo sin que el sistema lo detecte.
2. **Saturación de colas en RabbitMQ:** Si el Register Service o el Telemetry Service presentan lentitud o caídas, los mensajes se acumularán en las colas de RabbitMQ. Sin políticas de límite de cola (`max-length`), TTL o `dead-letter exchanges` bien configuradas, esto puede derivar en pérdida de datos o consumo excesivo de memoria en el broker.
3. **Escalabilidad del Call Service como cuello de botella:** El Call Service centraliza la distribución de todas las peticiones entrantes. En escenarios de alta concurrencia (múltiples dispositivos enviando datos simultáneamente) este servicio puede convertirse en un punto único de contención, especialmente si las validaciones contra Redis introducen latencia acumulada.
4. **Crecimiento no controlado en MongoDB:** Los registros del Register Service admiten payloads de formato variable, lo que puede derivar en documentos con estructuras muy heterogéneas y un crecimiento acelerado de la colección. Sin una política de retención de datos, índices adecuados o estrategia de archivado, las consultas del Telemetry Service degradarán su rendimiento con el tiempo.
5. **Detección de dispositivos desconectados basada en tiempo:** El mecanismo de `device.disconnected` depende de umbrales de tiempo para detectar inactividad. Si los intervalos de envío de un dispositivo son irregulares por naturaleza (conexiones intermitentes, redes inestables), el sistema puede generar falsos positivos de

desconexión, derivando en notificaciones innecesarias que reducen la confianza del usuario en las alertas.

Historias de usuario

ID	Historia de usuario
HU-01	Como usuario, quiero registrarme e iniciar sesión para acceder a la plataforma de manera segura.
HU-02	Como usuario, quiero registrar dispositivos IoT mediante su dirección MAC y nombre para administrarlos dentro de la plataforma.
HU-03	Como usuario, quiero consultar el estado y última conexión de mis dispositivos para verificar si están activos.
HU-04	Como dispositivo IoT, quiero enviar datos en formato JSON mediante HTTP para que sean almacenados por la plataforma.
HU-05	Como usuario, quiero visualizar los registros recientes enviados por mis dispositivos para monitorear su comportamiento.
HU-06	Como usuario, quiero solicitar reportes de telemetría para analizar los datos históricos de mis dispositivos.
HU-07	Como usuario, quiero recibir notificaciones cuando un dispositivo se desconecte o ocurra un evento importante.
HU-08	Como administrador, quiero consultar eventos y errores del sistema para supervisar el estado general de la plataforma.

Backlog

Este es un listado de las actividades generales a desarrollar para completar el proyecto, están ordenadas según el orden recomendado. Aun así, debido a la implementación de una arquitectura distribuida y con planeación de eventos y tablas, se es capaz de desarrollar algunas actividades sin desarrollar las anteriores.

ID	Backlog	Prioridad
BL-01	Configurar entorno base del proyecto con Docker Compose.	Alta
BL-02	Configurar PostgreSQL para usuarios y dispositivos.	Alta
BL-03	Implementar registro y gestión de dispositivos IoT.	Alta
BL-04	Configurar Redis para validación rápida de dispositivos.	Alta
BL-05	Configurar RabbitMQ para comunicación entre microservicios.	Alta
BL-06	Implementar almacenamiento de registros en MongoDB.	Alta
BL-07	Implementar consulta de registros recientes.	Media
BL-08	Implementar generación de reportes de telemetría.	Media
BL-09	Implementar detección automática de desconexión.	Media
BL-10	Implementar notificaciones por correo electrónico.	Media
BL-11	Implementar autenticación de usuarios con JWT.	Alta
BL-12	Implementar registro e inicio de sesión de usuarios.	Alta
BL-13	Implementar API Gateway y endpoints necesarios.	Alta
BL-14	Implementar recepción de datos desde dispositivos IoT.	Alta
BL-15	Desarrollar frontend para usuarios.	Alta
BL-16	Implementar panel básico de administración y monitoreo.	Media
BL-17	Documentar despliegue y ejecución local del sistema.	Alta

Endpoints

Todos los endpoints son accesibles a través del API Gateway. Los endpoints protegidos requieren un header `Authorization: Bearer <access_token>`, y los de administrador requieren adicionalmente que el token tenga `role: admin`.

Autenticación (/auth) Gestiona el registro, acceso y administración de usuarios y sesiones.

- POST /auth/signup registra un nuevo usuario en el sistema.
- POST /auth/login inicia sesión y retorna los tokens de acceso y refresco.
- POST /auth/logout revoca los tokens activos de la sesión.
- POST /auth/refresh renueva el token de acceso mediante el header `x-refresh-token`.
- GET /auth/me retorna el perfil del usuario autenticado.
- PATCH /auth/me actualiza el nombre o contraseña del usuario autenticado.

Dispositivos (/devices) Permite registrar y gestionar los dispositivos IoT vinculados a un usuario.

- POST /devices/register registra un nuevo dispositivo en el sistema de forma asíncrona.
- GET /devices/{user_id} retorna el dispositivo asociado a un usuario.
- PUT /devices/update actualiza el nombre o intervalo de reporte de un dispositivo.

Telemetría (/registers) Gestiona la recepción y consulta de los datos enviados por los dispositivos.

- POST /registers/received recibe y encola un registro de telemetría. Si el dispositivo estaba marcado como inactivo, se reactiva automáticamente.
- GET /registers/{device_id} retorna los registros almacenados de un dispositivo, ordenados por fecha descendente, con soporte de paginación.
- GET /registers/{device_id}/by-date filtra los registros por rango de fechas.

Reportes (/reports) Permite generar y consultar reportes a partir de la telemetría almacenada.

- POST /reports/{device_id}/generate solicita la generación asíncrona de un reporte para un dispositivo.

- GET /reports/{device_id} retorna los reportes generados, con métricas como valor máximo, valor más frecuente y porcentaje de cambio por variable.
- GET /reports/{device_id}/by-date filtra los reportes por rango de fechas.

Notificaciones (/notifications) Expone las notificaciones generadas por el sistema hacia el usuario.

- GET /notifications/me retorna todas las notificaciones del usuario autenticado, agregando las de todos sus dispositivos.
- GET /notifications/device/{device_id} retorna las notificaciones asociadas a un dispositivo específico.

La documentación completa de cada endpoint, incluyendo esquemas de request/response, parámetros y códigos de error, se encuentra en el repositorio oficial.

Pruebas

El proyecto cuenta con pruebas automatizadas ejecutadas a través de GitHub Actions y pruebas manuales realizadas con Postman. Las pruebas automatizadas se dividen en cuatro niveles: calidad de código con Ruff, pruebas de integración con cURL y pytest, y pruebas E2E que cubren el flujo completo a través del API Gateway. Adicionalmente se cuenta con una prueba específica para verificar el envío de notificaciones por correo mediante EmailUS. La documentación completa de cada caso de prueba se encuentra en el repositorio oficial.

Fallas simuladas

Se simularon dos escenarios de falla para verificar la resiliencia del sistema:

Reinicio del Redis de autenticación. Se detuvo el contenedor de Redis durante 2 minutos. La falla se detectó de tres formas: los endpoints autenticados devolvieron errores controlados al cliente, el endpoint /health reportó el estado degradado con HTTP 503, y el notification service registró automáticamente eventos críticos en MongoDB con la razón redis_connection_failed. Al restaurar el contenedor, los servicios recuperaron la conectividad de forma automática sin necesidad de reiniciarlos, y las sesiones activas se mantuvieron intactas gracias a la persistencia de Redis. Los tokens se limpian solos al vencer mediante TTL, sin intervención manual.

Caída del Device Service. Se detuvo manualmente el contenedor del servicio de dispositivos. La falla se detectó porque los endpoints que dependen de él directamente (GET /devices/me) devolvieron errores controlados, el call service publicó eventos

system.error a RabbitMQ con razón upstream_error, y el notification service los persistió automáticamente en MongoDB. En contraste, los endpoints asíncronos (POST /devices/register, POST /registers/received) continuaron operando con normalidad a través de RabbitMQ. Al recuperar el servicio, los eventos encolados durante la caída fueron procesados correctamente y ambos dispositivos quedaron disponibles, demostrando el desacoplamiento asíncrono del sistema.

Repositorio oficial

El proyecto puede ser accedido a través del repositorio oficial en Github en el siguiente enlace:

<https://github.com/JoseLuisAlvarezManica/loteur>

Página web en Vercel

La página web puede ser accedido a través del siguiente enlace antes de ser descontinuada:

<https://ioteur.vercel.app/>